

Содержание

1	Короткая документация по реализации	1
1.1	1. Алгоритм МиниГОСТ	1
1.2	2. Алгоритм расчёта Δ_{pi}	4
1.3	3. Как всё используется в проекте	5
1.4	4. Короткий вывод	6
2	Начинаем восстанавливать ключи	6

1 Короткая документация по реализации

Ниже описано только то, что подтверждается текущим кодом проекта.

Основные источники: - `src/minigost.c` - `include/minigost.h` - `tools/prepare_phase.py` - `tools/generate_lookup_table.py` - `tools/compute_delta_pi.c`
- `Makefile`

1.1 1. Алгоритм МиниГОСТ

1.1.1 1.1. Формат данных

По текущему коду: - блок: 16 бит; - полублок: 8 бит; - ключ: 32 бита; - ключ передаётся как `const uint8_t key[4]`; - число раундов: 12.

Публичный API:

```
uint16_t minigost_encrypt_block(uint16_t plaintext, const uint8_t key[4]);
uint16_t minigost_decrypt_block(uint16_t ciphertext, const uint8_t key[4]);
int minigost_encrypt_ecb(const uint8_t *plaintext, uint8_t *ciphertext, size_t length, const uint8_t key[4]);
int minigost_decrypt_ecb(const uint8_t *ciphertext, uint8_t *plaintext, size_t length, const uint8_t key[4]);
```

1.1.2 1.2. Интерпретация ключа

Ключ понимается так:

$K = (K^{(4)}, K^{(3)}, K^{(2)}, K^{(1)})$

где: - $K^{(4)}$ — старший байт; - $K^{(1)}$ — младший байт.

Соответствие массиву `key[4]`:

```
key[0] =  $K^{(4)}$ 
key[1] =  $K^{(3)}$ 
key[2] =  $K^{(2)}$ 
key[3] =  $K^{(1)}$ 
```

Пример:

```
uint8_t key[4] = {0xA1, 0xB2, 0xC3, 0xD4};
```

означает:

```
 $K^{(4)}$  = 0xA1
 $K^{(3)}$  = 0xB2
 $K^{(2)}$  = 0xC3
 $K^{(1)}$  = 0xD4
```

1.1.3 1.3. Как ключ используется в раундах

В коде зашифто расписание:

```
static const uint8_t schedule[12] = {3, 2, 1, 0, 3, 2, 1, 0, 0, 1, 2, 3};
```

Это даёт порядок раундовых ключей:

```
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$ 
 $K^{(1)}, K^{(2)}, K^{(3)}, K^{(4)},$ 
 $K^{(4)}, K^{(3)}, K^{(2)}, K^{(1)}$ 
```

То есть по раундам:

```
r1 =  $K^{(1)}$ 
r2 =  $K^{(2)}$ 
r3 =  $K^{(3)}$ 
```

```

r4  = K^(4)
r5  = K^(1)
r6  = K^(2)
r7  = K^(3)
r8  = K^(4)
r9  = K^(4)
r10 = K^(3)
r11 = K^(2)
r12 = K^(1)

```

1.1.4 1.4. Подготовительная функция

В `src/minigost.c` нет вычисления `prep_phase` на лету. Вместо этого используется таблица:

```
static const uint8_t PREP_PHASE_TABLE[256] = { ... };
```

Раундовая функция:

```
static inline uint8_t round_function(uint8_t n1, uint8_t k) {
    uint8_t sum = (n1 + k) & 0xFF;
    return PREP_PHASE_TABLE[sum];
}
```

То есть:

```
F(x, k) = PREP_PHASE_TABLE[(x + k) mod 256]
```

1.1.5 1.5. Как сгенерирован PREP_PHASE_TABLE

Генератор: `tools/prep_phase.py`.

В нём заданы две 4-битные подстановки:

```

p1 = [12, 4, 6, 2, 10, 5, 11, 9, 14, 8, 13, 7, 0, 3, 15, 1]
p0 = [6, 8, 2, 3, 9, 10, 5, 12, 1, 14, 4, 7, 11, 13, 0, 15]

```

Для каждого байта `byte` от 0 до 255 делается:

```

high_nibble = (byte >> 4) & 0xF
low_nibble = byte & 0xF

high_sub = p1[high_nibble]
low_sub = p0[low_nibble]

substituted = (high_sub << 4) | low_sub
result = ((substituted << 5) | (substituted >> 3)) & 0xFF

```

То есть:

```

h = старшие 4 бита byte
l = младшие 4 бита byte

s(byte) = (p1[h] << 4) | p0[l]
T(byte) = ROTL8(s(byte), 5)

```

Где:

```
ROTL8(x, 5) = ((x << 5) | (x >> 3)) & 0xFF
```

После этого генератор просто печатает все 256 значений подряд в C-массив.

Пример из кода:

```

byte = 0x93 = 10010011b
h = 9
l = 3
p1[9] = 8
p0[3] = 3
substituted = 0x83
result = ROTL8(0x83, 5) = 0x70

```

То же значение проверяется встроенным тестом в `src/minigost.c`.

1.1.6 1.6. Один раунд шифрования

Блок делится на два байта:

L = старший байт

R = младший байт

Код раунда:

```
uint8_t n1 = (*block >> 8) & 0xFF;
uint8_t n2 = *block & 0xFF;
uint8_t g = round_function(n2, key_element);
uint8_t new_n2 = n1 ^ g;
*block = ((uint16_t)n2 << 8) | new_n2;
```

Это соответствует преобразованию:

$$(L, R) \rightarrow (R, L \text{ XOR } F(R, k))$$

Это сеть Фейстеля.

1.1.7 1.7. Полное шифрование блока

Код:

```
for (int round = 0; round < 12; round++) {
    uint8_t key_element = key[schedule[round]];
    minigost_round(&block, key_element);
}
```

```
uint8_t n1 = (block >> 8) & 0xFF;
uint8_t n2 = block & 0xFF;
block = ((uint16_t)n2 << 8) | n1;
```

То есть:

1. Выполнить 12 раундов по расписанию ключей.
2. После 12-го раунда поменять местами левый и правый полублок.

Если обозначить вход как (L_0, R_0) , то каждый раунд:

$$(L(i+1), R(i+1)) = (R(i), L(i) \text{ XOR } F(R(i), \text{round_key}(i)))$$

После последнего раунда:

$$\text{ciphertext} = \text{swap}(L_{12}, R_{12})$$

1.1.8 1.8. Расшифрование блока

Код:

```
uint8_t n1 = (block >> 8) & 0xFF;
uint8_t n2 = block & 0xFF;
block = ((uint16_t)n2 << 8) | n1;
```

```
for (int round = 11; round >= 0; round--) {
    uint8_t key_element = key[schedule[round]];
    minigost_round_inv(&block, key_element);
}
```

Обратный раунд:

```
uint8_t n1 = (*block >> 8) & 0xFF;
uint8_t n2 = *block & 0xFF;
uint8_t g = round_function(n1, key_element);
uint8_t new_n1 = n2 ^ g;
*block = ((uint16_t)new_n1 << 8) | n1;
```

То есть:

$$(L, R) \rightarrow (R \text{ XOR } F(L, k), L)$$

Порядок действий:

1. Сначала делается обратная финальная перестановка.
2. Затем идут 12 обратных раундов в обратном порядке ключей.

1.1.9 1.9. ECB режим

Код собирает 16-битный блок из двух байтов в big-endian виде:

```
uint16_t block = ((uint16_t)plaintext[i] << 8) | plaintext[i + 1];
```

Каждый блок шифруется независимо:

```
Cj = Enc_K(Pj)
```

Ограничение: - длина входа должна быть кратна 2; - иначе функция возвращает -1.

1.2 2. Алгоритм расчёта delta_pi

1.2.1 2.1. Что считается функцией pi

В tools/compute_delta_pi.c шифрование не вычисляется на лету. Подключается готовая lookup-таблица:

```
#include "lookup_tables/DEADBEEF_lookup.h"
```

Значит:

```
pi(x) = MINIGOST_LOOKUP_TABLE[x]
```

для всех x от 0 до 65535.

1.2.2 2.2. Пространство перебора

По коду: - $N = 65536 = 2^{16}$; - перебираются все β от 1 до 65535; - для каждого β перебираются все α от 1 до 65535.

Нулевые маски не учитываются.

1.2.3 2.3. Вектор для фиксированного beta

Код:

```
for (int x = 0; x < 65536; x++) {
    uint16_t y = lookup_table[x];
    f[x] = parity16(beta & y) ? -1 : 1;
}
```

Здесь $\text{parity}_{16}(z)$ — чётность числа единиц в z .

То есть строится вектор:

```
f_beta(x) = 1, если parity(beta & pi(x)) = 0
f_beta(x) = -1, если parity(beta & pi(x)) = 1
```

или короче:

```
f_beta(x) = (-1)^(<beta, pi(x)>)
```

1.2.4 2.4. Walsh-Hadamard transform

Код:

```
for (int len = 1; len < 65536; len <= 1) {
    for (int start = 0; start < 65536; start += (len < 1)) {
        for (int i = 0; i < len; i++) {
            int a = f[start + i];
            int b = f[start + i + len];
            f[start + i] = a + b;
            f[start + i + len] = a - b;
        }
    }
}
```

После этого $f[\alpha]$ — коэффициент Уолша для маски α .

1.2.5 2.5. Корреляция и delta_pi

Код:

```
const float norm = 1.0f / 65536.0f;
float correlation = -(float)f[alpha] * norm;
float abs_correlation = fabsf(correlation);
```

То есть:

```
c(alpha, beta) = -f[alpha] / 65536
delta_pi = max |c(alpha, beta)|
```

При нахождении нового максимума сохраняются: - alpha_max; - beta_max; - correlation_value; - max_correlation.

1.2.6 2.6. Что делает программа пошагово

1. Берёт готовую lookup-таблицу pi.
2. Для каждого beta != 0 строит вектор длины 65536.
3. Применяет FWT.
4. Для каждого alpha != 0 вычисляет корреляцию.
5. Ищет максимальный модуль корреляции.
6. Печатает результат и сохраняет его в delta_pi_result.txt.

1.3 3. Как всё используется в проекте

1.3.1 3.1. Базовая реализация шифра

Основные файлы: - src/minigost.c - include/minigost.h

Проверка:

```
make test
```

1.3.2 3.2. Использование как C-библиотеки

Сборка библиотек:

```
make lib/libminigost.a
make lib/libminigost.dylib # macOS
```

Пример интеграции:

```
make example
```

1.3.3 3.3. Python-обёртка над C

Запуск:

```
make python
```

1.3.4 3.4. Генерация lookup-таблицы

Таблица строится так:

```
for block in range(0x10000):
    encrypted = cipher.encrypt_block(block, key)
    lookup_table.append(encrypted)
```

То есть материализуется полное отображение:

```
x -> pi(x)
```

Поддерживаются форматы: - .bin - .h - .md - .py

Запуск:

```
python3 tools/generate_lookup_table.py --key 0xDEADBEEF
./generate_lookup.sh
```

1.3.5 3.5. Расчёт delta_pi

Запуск:

```
./run_delta_pi.sh
```

Что делает скрипт: 1. Проверяет наличие lookup_tables/DEADBEEF_lookup.h. 2. Компилирует tools/compute_delta_pi.c. 3. Запускает build/compute_delta_pi. 4. Перекладывает delta_pi_result.txt в results/.

Важно: tools/compute_delta_pi.c сейчас жёстко подключает конкретный header-файл, так что без смены #include он считает delta_pi только для уже выбранного ключа.

1.3.6 3.6. Проверка корректности

Полное сравнение Python и C по всем 65536 блокам:

```
make compare
```

Смысл проверки:

```
blocks = range(0x10000)
py_results = run_python_batch(cipher, mode, py_key, blocks)
c_results = run_c_batch(runner, mode, c_key, blocks)
```

1.3.7 3.7. Минимальный набор рабочих сценариев

```
make test
make example
make benchmark
make compare
make python
python3 tools/generate_lookup_table.py --key 0xDEADBEEF
./generate_lookup.sh
./run_delta_pi.sh
python3 verify_correlation.py
sage verify_in_sage.py
./batch_compute_delta_pi.sh 10
```

1.4 4. Короткий вывод

Сейчас проект состоит из трёх основных частей:

1. Реализация МиниГОСТ как 12-раундовой сети Фейстеля над 16-битным блоком и 32-битным ключом.
2. Генерация полной lookup-таблицы $\pi : \{0,1\}^{16} \rightarrow \{0,1\}^{16}$ для фиксированного ключа.
3. Вычисление $\text{delta_pi} = \max |c(\alpha, \beta)|$ через полный перебор масок и FWT.

2 Начинаем восстанавливать ключи

заметим, что $H_i^{-1} = TH_iT$ а значит справедлива следующая запись:

$$H_1H_2H_3H_4H_1H_2H_3H_4H_4H_3T = H_1H_2H_3H_4H_1H_2H_3H_4H_4H_3H_2H_1T(TH_1T)(TH_2T)$$

Теперь

Переберем $N = 10^3$ ключей

$\forall K = (K_1, K_2, K_3, K_4) :$ Вычислим $\overline{F_k} = H_1H_2H_3H_4H_1H_2H_3H_4H_4H_3$ (через K обозначим истинный ключ)

находим $\alpha, \beta \in V_{16} : |\delta_{\alpha, \beta}^{\overline{F_k}}| = \max = \delta^{\overline{F_k}}$

для данной α, β и ключа K достраиваем до полного шифра: $E_k = \overline{F_k}H_2H_1T$

для E_k строим таблицу переходов (массив $p[]$) из 2^{16} адресов, где по адресу $x \in \overline{0, 2^{16} - 1}$ находится $E_k(x)$

Далее ____

$\forall K' = (K'_1, K'_2) \in V_{16}$

S := 0

$\forall x \in \overline{0, \frac{M}{(\delta^{\overline{F_k}})^2}} \approx 2^{14}$, где $M \in \overline{1, 10}$

y := p[x]

Если $(\alpha, x) = (\beta, TH_1H_2T)$, то S++

$S := \frac{S}{2^{16}}$

$\bar{\delta} = 2S - 1$

тут $(x, y) = + \sum_{i=1}^{16} x_i y_i$ - сумма по модулю 2

Ожидаемый результат в теории:

- Если $K' = (K1, K2)$ - истинный, то $\bar{\delta} = \delta^{F_k}$

- Если ложный, то $\bar{\delta} \approx 0$

А что будет у нас: - Если истинный, то $\bar{\delta} \approx \delta^{F_k}$ - Если ложный, то ? - надо выяснить, насколько это близко к δ^{F_k}